Multi-Tenant Inventory & Order Fulfillment (Prototype)

Candidate Screening Project — Level 3 (~2 years experience) · Prototype scope

1. Overview

Take the inventory concept further with the architectural depth of a real small SaaS: multiple tenants, multiple warehouses per tenant, and safe concurrent order fulfillment. Still zero external APIs — the depth here is in tenant isolation and concurrency correctness, not integrations.

2. Functional Requirements

1. **Multi-tenancy:** each tenant (business) has its own isolated data — products, warehouses, orders. A user belongs to exactly one tenant.
2. **Tenant onboarding:** a simple “create tenant” flow for a new business to sign up (no billing required).
3. **Multiple warehouses per tenant:** stock is tracked per product per warehouse, same movement-history model as a standard inventory system (see Brief D for the base pattern if useful).
4. **Warehouse-to-warehouse transfer:** moving stock between two warehouses of the same tenant creates a single transfer record (not two disconnected +/− movements that could get out of sync).
5. **Order fulfillment with atomic reservation:** creating an order reserves stock for its line items. Under concurrent orders competing for the last few units of a low-stock item, the system must never oversell — exactly one of the competing orders should succeed for the last unit, and the other should get a clear “insufficient stock” response.
6. **Scheduled reconciliation:** a cron job that runs periodically and flags: (a) stale/low-stock items per tenant, and (b) any inventory count that doesn't match its movement history (a real drift check, not a cosmetic one).

3. Explicitly Out of Scope

- Payment processing
- Real courier/shipping API integration
- A customer-facing storefront (this is an internal ops tool)
- Tenant billing/subscription management

> *Building any of the above will not earn extra credit — it takes time away from getting the in-scope requirements genuinely solid. Scope judgment is itself part of what's being evaluated.*

4. Non-Functional Requirements

- **Provable tenant isolation:** enforce isolation with Row-Level Security at the database layer, not application-level filtering — a bug in a query's WHERE clause must not be able to leak another tenant's data. Test this adversarially: try to fetch another tenant's data directly and confirm RLS blocks it.
- **Atomic stock reservation:** order fulfillment must use a proper database transaction or constraint (e.g. a conditional update that fails if stock would go negative) — a “check stock, then write order” done as two separate steps in application code is a race condition and will be tested with concurrent requests.
- **Idempotent, safe-to-repeat reconciliation:** running the reconciliation job twice in a row must not create duplicate flags or corrupt data.
- **Transfer consistency:** a warehouse transfer must be all-or-nothing — never leave stock debited from the source warehouse without being credited to the destination.

5. Suggested Tech Stack

Versions below were verified current as of July 2026. Substituting equivalents is fine — justify the choice in your README.

- **Framework:** `Next.js 16.2` (App Router, current Active LTS as of July 2026 — do not scaffold on 15.x or the 16.3 preview) with `TypeScript 6.0.3`.
- **Runtime:** `Node.js 24` (current Active LTS). Node 22 is Maintenance-only; don't start a new project on it.
- **Styling:** `Tailwind CSS v4`.
- **Validation:** `Zod 4.4.3` for all form and API-boundary validation — never trust client input server-side.
- **Database + Auth:** Supabase — provision on `Postgres 17` (Postgres 14 was deprecated by Supabase on July 1, 2026 and will no longer accept new projects). Use the Supabase CLI (`v2.109+`) for local dev + migrations.
- **Deployment:** Vercel (free tier is enough).
- **Multi-tenancy:** a `tenant_id` column on every tenant-owned table, enforced via Postgres RLS policies keyed to the authenticated user's tenant.
- **Atomic reservation:** a single SQL statement (e.g. `UPDATE ... WHERE stock >= quantity` checked via row count, or a Postgres advisory lock around the reservation transaction) — not separate read-then-write calls from the application.
- **Scheduled jobs:** `Vercel Cron` or a Supabase scheduled Edge Function for the reconciliation sweep.

6. Task List

Phase 0 — Setup

- ☐ Scaffold Next.js 16.2 + TypeScript, Tailwind v4
- ☐ Supabase project on Postgres 17

Phase 1 — Tenancy & Data Model

- ☐ Schema: tenants, users (tenant-scoped), warehouses, products, stock_movements, orders, order_items, transfers
- ☐ RLS policies on every tenant-owned table
- ☐ Adversarial test: confirm a logged-in user cannot read/write another tenant's rows via direct API calls

Phase 2 — Inventory Core

- ☐ Per-warehouse stock tracking via movement history (append-only, same discipline as Brief D)
- ☐ Warehouse-to-warehouse transfer as a single atomic operation

Phase 3 — Order Fulfillment

- ☐ Order creation with atomic stock reservation (DB-enforced, not app-level check-then-write)
- ☐ Concurrency test: fire multiple simultaneous orders for the last unit of a low-stock item and confirm exactly one succeeds

Phase 4 — Reconciliation & Deploy

- ☐ Scheduled reconciliation job: flags low-stock and any stock/history mismatch
- ☐ Confirm the job is idempotent (safe to run twice)
- ☐ Deploy to Vercel; write README

7. Submission Requirements

- Live URL
- GitHub repo
- README covering: the tenant isolation approach, and exactly how atomic stock reservation is implemented
- A short note on what you'd build next with another week

8. README Must Also Answer

Explain how you enforce tenant isolation at the database level (not just the UI), and walk through exactly what prevents overselling when two orders compete for the same last unit of stock.